PIC 10A 1A

TA: Bumsu Kim

This content is protected and may not be shared, uploaded, or distributed.

The author does not grant permission for these notes to be posted anywhere without prior consent.

UCLA

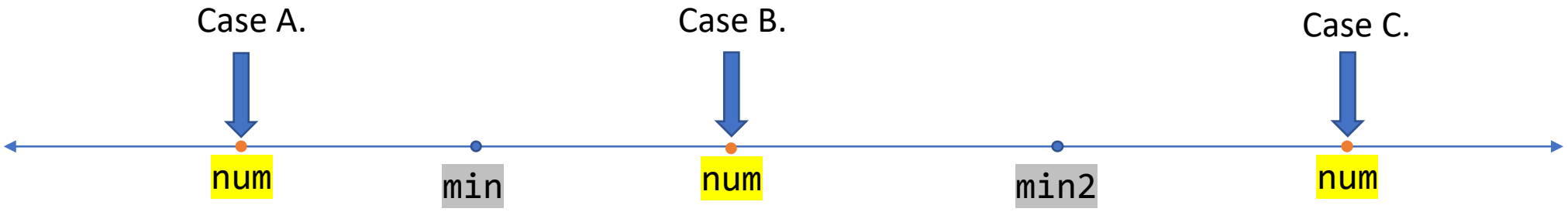
Today…

- HW3 Hints
- Functions

HW3 Hints

HW3 Hints

- Exercise 2: Find the 2nd smallest number in the given stream
  - Keep track of the smallest, and the 2nd smallest (say, `min` and `min2`, respectively)
  - When a new input `num` is given, there are three cases:



  - Case C is the easiest (Do we need to update something?)
  - In Case B, which variable(s) should we update? Does something stay the same in this case?
  - How about Case A?

UCLA

HW3 Hints

- If you figured out how to handle those three cases, let's think about how to handle the "same number" cases

- Case A' : num == min
- Case B' : num == min2

- What do we have to do in these cases?

HW3 Hints

- To sum up, the pseudocode may look like:

```
while ( /* input is valid */ ) {
    if ( /* case A */) {
        // do something
        // handle case A' if needed
    }
    else if (/* case B */) {
        // do something
        // handle case A' and/or B' if needed
    }
    else { // case C
        // do anything you need
    }
}
```

- And finally, give some *special care* for “single input” case (e.g. the last ex. in the HW instr.)

UCLA

Functions

Functions

- Functions can be
  - Declared and then defined later
    - If you declare the function first (not define it right now) then need ;
  - Defined when it is declared (so both are done at the same time)

- The declaration determines a “signature” of the function
  - [Return Type] [Function Name] [Input Parameters] (and other options later)

```
double find_max(double a, double b);
```

UCLA

Functions

- The declaration determines a “signature” of the function
  - [Return Type] [Function Name] [Input Parameters] (and other options later)

```
double find_max(double a, double b);
```

- Return Type
  - The type of the expression returned by the function
  - If the function returns nothing, can be `void`
- Function Name
  - Name of the function; cannot be a reserved word, the same naming rule applies with variables
- Input Parameters
  - The objects passed to the function
  - Number of arguments can be 0, 1, or more
  - Still need parentheses () even when the function gets zero parameters

UCLA

Functions – Examples

- A function returning the maximum of two numbers

```
double find_max(double a, double b) {
        return (a > b) ? a : b;
}
```

Recall: The ternary operator

```
A ? B : C
```

is (almost) equivalent to

```
if (A) { B; }
else {C;}
```

- A ***procedure*** is a function returning `void`
  - Which means, it doesn't return anything
  - Example: a function printing the max to the console

```
void print_max(double a, double b) {
    cout << "The max of " << a << " and " << b
         << " is " << find_max(a, b);
}
```

Possible if `find_max` is ***declared*** before this expression

Q: [`true` or `false`?]
Using "`return`" keyword in a ***procedure*** results in a syntax error.

UCLA

Functions – Examples

- A function that inputs several different types of arguments

```
void func(unsigned int i, string str) {
        cout << str[i];
}
```

Here `i` cannot be negative

- A ***predicate*** function is a function returning `bool`
  - The following function checks whether the first letter of the string is capitalized or not

```
bool isCapitalized(string str) {
    if ('A' <= str[0] && str[0] <= 'Z') {
        return true;
    }else {
        return false;
    }
}
```

True if `str[0]` is between 'A' and 'Z' (otherwise it is not a capital letter)

UCLA

Exercise – Collatz Conjecture

- Given a positive integer, perform the following operation each step:
  - If the number is even, divide it by two $x \rightarrow x/2$
  - If the number is odd, triple it and add one $x \rightarrow 3x + 1$
- The Collatz conjecture states that you will always reach 1.
- Write a program that computes how many steps it takes to reach 1 starting from a positive integer. The input and output should be:

```
Starting from X.
Step 1: X
Step 2: X
...
Step X: X
Starting from X, it took X steps.
```

Output example starting from 6

```
Starting from 6
Step 1: 3
Step 2: 10
Step 3: 5
Step 4: 16
Step 5: 8
Step 6: 4
Step 7: 2
Step 8: 1
Starting from 6, it took 8 Steps.
```

- Hints
  - Again, the `while` loop will be useful
  - This function is a ***procedure***, i.e., it will return `void`

UCLA

Your Feedback is welcome

- Don’t hesitate to give a feedback on the discussion
- You can use a link in BruinLearn (Google Form)

Click this link

At this point, you may want to request
“more hw hints”,
“more challenging examples/exercises”,
“more details on the classroom examples”, etc.
Please let me know what you want!